□ Heat Pump Feasibility & Simulation Software: MVP Specification

---

□ PART 1: DETAILED MVP SPECIFICATION

□ 1. MVP Overview

1.1 Purpose

Develop a **web-based MVP** for heat pump feasibility studies and basic simulation, targeting **residential/commercial buildings** (greenhouse/CEA support deferred to Phase 2). The MVP will: - Enable **non-experts** (architects, contractors, homeowners) to **size and evaluate** heat pump systems. - Provide **quick feasibility checks** with thermal and economic outputs. - Serve as a **foundation** for future expansion (greenhouses, hybrid systems, advanced simulation).

1.2 Goals

| Goal | Metric | Target | |------|--------|--------| | **User Adoption** | Active users | 1,000 in 6 months | | **Simulation Accuracy** | Match EnergyPlus/HPDM within ±5% | Validated via benchmarks | | **Time-to-Result** | From input to feasibility report | < 5 minutes | | **Ease of Use** | User satisfaction (survey) | ≥ 4.5/5 | | **Performance** | Simulation runtime | < 10 seconds |

1.3 Scope (Included in MVP)

□ **User Inputs** - Building dimensions (area, volume, orientation) - Envelope materials (walls, roof, floor, windows) - Location (auto-climate data) - Desired indoor temperature - Heat pump type (air-source, ground-source)

□ **Core Calculations** - Heat loss (BS EN 12831 or ASHRAE simplified) - Heat pump sizing (capacity, COP/SCOP) - Basic economic analysis (CAPEX, OPEX, simple payback)

□ **Outputs** - Heat load profile (kW) - Recommended heat pump size (kW) - Estimated COP/SCOP - Simple payback period (years) - Carbon savings (vs. gas boiler)

 Database - Material properties (U-values, thermal conductivity) - Climate data (temperature, humidity) - Heat pump performance curves (generic models)

 UI/UX - Web-based (responsive) - Guided input workflow - Visual results (charts, heat loss map)

1.4 Scope (Excluded from MVP)

 Greenhouse/CEA mode (Phase 2) **Hybrid systems** (solar + heat pump + storage) (Phase 2) **Dynamic simulation** (hourly/transient) (Phase 3) **Advanced economics** (NPV, IRR, incentives) (Phase 2) **Collaboration features** (Phase 3) **BIM/Revit integration** (Phase 4) **IoT monitoring** (Phase 4)

---

2. User Stories & Requirements

2.1 User Personas

| Persona | Description | Pain Points | MVP Use Case |
|---|---|---|---| | **Architect** | Designs residential/ commercial buildings | Needs **quick heat pump sizing** for early-stage designs | Input building specs → get heat pump size + feasibility | | **HVAC Contractor** | Installs heat pump systems | Wants **fast, accurate quotes** for clients | Input building details → generate proposal with ROI | | **Homeowner** | DIY heat pump research | Overwhelmed by **technical complexity** | Step-by-step guide → understand if heat pump is viable | | **Energy Consultant** | Advises on efficiency upgrades | Needs **defensible calculations** for clients | Run scenarios → compare heat pump vs. gas boiler |

2.2 Functional Requirements

FR-1: User Inputs

| ID | Requirement | Priority | Notes | |---|---|---|---| | FR-1.1 | Building dimensions (length, width, height, floors) | **P0** | Metric units (m) + imperial (ft) | | FR-1.2 | Envelope materials (walls, roof, floor, windows) | **P0** | Dropdown selection + custom entry | | FR-1.3 | Window/door specifications (area, type, U-value) | **P0** | Glazing type (single/double/triple) | | FR-1.4 | Location (city/country or coordinates) | **P0** | Auto-fill climate data | | FR-1.5 | Desired indoor temperature (°C/°F) | **P0** | Default: 20°C (68°F) | | FR-1.6 | Heat pump type (air-source, ground-source) | **P0** | Ground-source: horizontal/vertical loop | | FR-1.7 | Existing heating system (gas boiler, electric, etc.) | **P1** | For comparison in economics | | FR-1.8 | Occupancy (people, usage hours) | **P2** | Affects internal gains |

FR-2: Calculations & Simulation

| ID | Requirement | Priority | Notes | |----|-------------|----------|-------| | FR-2.1 | Heat loss calculation (BS EN 12831 simplified) | **P0** | Wall, roof, floor, windows, ventilation | | FR-2.2 | Heat pump sizing (kW) | **P0** | Based on peak heat load | | FR-2.3 | COP/SCOP estimation | **P0** | Dynamic vs. outdoor temperature | | FR-2.4 | Annual energy demand (kWh) | **P0** | Heating + cooling (if applicable) | | FR-2.5 | Heat pump runtime estimation | **P1** | Hours/year at full/partial load | | FR-2.6 | Defrost energy (for air-source) | **P2** | Climate-dependent |

FR-3: Economic Analysis

| ID | Requirement | Priority | Notes | |----|-------------|----------|-------| | FR-3.1 | CAPEX estimation (heat pump + installation) | **P0** | Regional cost databases | | FR-3.2 | OPEX estimation (electricity cost) | **P0** | Local energy prices (API) | | FR-3.3 | Simple payback period (years) | **P0** | (CAPEX / Annual Savings) | | FR-3.4 | Carbon savings (kg CO_2/year) | **P0** | vs. gas boiler (default) | | FR-3.5 | Energy cost comparison (heat pump vs. existing) | **P1** | Annual savings |

FR-4: Outputs & Visualization

| ID | Requirement | Priority | Notes | |----|-------------|----------|-------| | FR-4.1 | Summary report (PDF/printable) | **P0** | Key metrics + recommendations | | FR-4.2 | Heat loss breakdown (chart) | **P0** | By component (walls, roof, etc.) | | FR-4.3 | Heat pump performance curve | **P0** | COP vs. outdoor temperature | | FR-4.4 | Monthly energy demand (chart) | **P1** | Seasonal variation | | FR-4.5 | Interactive 3D heat loss map | **P2** | Color-coded by surface |

FR-5: Database

| ID | Requirement | Priority | Notes | |----|-------------|----------|-------| | FR-5.1 | Material database (U-values, λ, ρ, c_p) | **P0** | CIBSE/ASHRAE defaults | | FR-5.2 | Climate database (temperature, humidity) | **P0** | Meteonorm or OpenWeatherMap | | FR-5.3 | Heat pump performance curves | **P0** | Generic models (3–5 types) | | FR-5.4 | Regional cost data (installation, energy) | **P1** | Country/state-level | | FR-5.5 | Carbon intensity factors | **P1** | g CO_2/kWh (electricity + gas) |

FR-6: System & UX

| ID | Requirement | Priority | Notes | |----|-------------|----------|-------| | FR-6.1 | Responsive web app (desktop + mobile) | **P0** | Next.js or Streamlit | | FR-6.2 | Guided input workflow (step-by-step) | **P0** | Progress bar, tooltips | | FR-6.3 | Save/load projects (local storage) | **P0** | No cloud in MVP | | FR-6.4 | Unit toggle (Metric/Imperial) | **P0** | Consistent across app | | FR-6.5 | Dark/light mode | **P2** | User preference | | FR-6.6 | Export data (CSV/JSON) | **P1** | For further analysis |

🏗 3. System Architecture

3.1 High-Level Architecture

```
mermaid graph TD A[Client: Next.js] -->|API Calls| B[Backend: FastAPI] B --> C[Simulation Engine: Python] B --> D[Database: PostgreSQL] D --> E[Material Properties] D --> F[Climate Data] D --> G[Heat Pump Models] C --> H[Calculation Modules] H --> H1[Heat Loss] H --> H2[Heat Pump Sizing] H --> H3[Economic Analysis] A --> I[Visualization: Plotly.js] B --> J[External APIs] J --> J1[OpenWeatherMap] J --> J2[Energy Price APIs]
```

3.2 Component Diagram

```
mermaid classDiagram class Frontend { +Next.js (React) +UI Components +State Management (Zustand) +Form Handling (React Hook Form) } class Backend { +FastAPI +RESTful Endpoints +Authentication (JWT) } class SimulationEngine { +HeatLossCalculator +HeatPumpSizer +EconomicAnalyzer +DataValidator } class Database { +PostgreSQL +MaterialTable +ClimateTable +HeatPumpTable } class ExternalAPIs { +OpenWeatherMap +EnergyPriceAPI } Frontend --> Backend : HTTP Requests Backend --> SimulationEngine : Calls Backend --> Database : Queries Backend --> ExternalAPIs : Fetches Data SimulationEngine --> Database : Reads
```

3.3 Data Flow

1. **User Inputs** → Frontend validates → Sent to Backend (JSON).
2. **Backend** validates → Calls **Simulation Engine**.
3. **Simulation Engine**:
    - Fetches **material/climate data** from Database.
    - Runs **heat loss calculation**.
    - Sizes **heat pump** and estimates **COP/SCOP**.
    - Calculates **economics** (CAPEX, OPEX, payback).
4. **Backend** returns **results** (JSON) → Frontend renders.
5. **Frontend** displays **charts, tables, report**.

🗄 4. Data Model

4.1 Database Schema (PostgreSQL)

```sql -- Users (for future auth, MVP uses local storage) CREATE TABLE users ( id SERIAL PRIMARY KEY, email VARCHAR(255) UNIQUE, password*hash VARCHAR(255), created*at TIMESTAMP DEFAULT NOW() );

-- Projects (MVP: stored in localStorage; Phase 2: DB) CREATE TABLE projects ( id SERIAL PRIMARY KEY, user*id INTEGER REFERENCES users(id), name VARCHAR(255), building*type VARCHAR(50), -- residential,
```

commercial created*at TIMESTAMP DEFAULT NOW(), updated*at TIMESTAMP DEFAULT NOW());

-- Building Dimensions CREATE TABLE building*dimensions (id SERIAL PRIMARY KEY, project*id INTEGER REFERENCES projects(id), length FLOAT, -- meters width FLOAT, height FLOAT, floors INTEGER, orientation VARCHAR(20) -- N, S, E, W);

-- Materials CREATE TABLE materials (id SERIAL PRIMARY KEY, name VARCHAR(100), description TEXT, thermal*conductivity FLOAT, -- W/m·K density FLOAT, -- kg/m³ specific*heat FLOAT, -- J/kg·K u_value FLOAT, -- W/m²·K (pre-calculated) type VARCHAR(50) -- wall, roof, floor, window);

-- Building Surfaces (walls, roof, floor, windows) CREATE TABLE surfaces (id SERIAL PRIMARY KEY, project*id INTEGER REFERENCES projects(id), material*id INTEGER REFERENCES materials(id), surface_type VARCHAR(50), -- wall, roof, floor, window area FLOAT, -- m² thickness FLOAT -- m (for custom materials));

-- Windows CREATE TABLE windows (id SERIAL PRIMARY KEY, project*id INTEGER REFERENCES projects(id), glazing*type VARCHAR(50), -- single, double, triple area FLOAT, -- m² u*value FLOAT, -- W/m²·K shading*factor FLOAT DEFAULT 1.0);

-- Locations CREATE TABLE locations (id SERIAL PRIMARY KEY, city VARCHAR(100), country VARCHAR(100), latitude FLOAT, longitude FLOAT, climate_zone VARCHAR(50));

-- Climate Data (cached from API) CREATE TABLE climate*data (id SERIAL PRIMARY KEY, location*id INTEGER REFERENCES locations(id), month INTEGER, -- 1-12 avg*temp FLOAT, -- °C min*temp FLOAT, max_temp FLOAT, humidity FLOAT -- %);

-- Heat Pump Models CREATE TABLE heat*pump*models (id SERIAL PRIMARY KEY, name VARCHAR(100), type VARCHAR(50), -- air-source, ground-source heating*capacity*min FLOAT, -- kW heating*capacity*max FLOAT, cop*at*7C FLOAT, -- COP at 7°C outdoor cop*at*35C FLOAT, -- COP at 35°C outdoor efficiency_curve TEXT -- JSON: {temp: cop, ...});

-- Economic Data CREATE TABLE economic*data (id SERIAL PRIMARY KEY, country VARCHAR(100), electricity*price FLOAT, -- $/kWh gas*price FLOAT, -- $/kWh heat*pump*install*cost FLOAT, -- $/kW carbon*intensity*electricity FLOAT, -- g CO_2/kWh carbon*intensity*gas FLOAT -- g CO_2/kWh);

-- Project Results (cached for performance) CREATE TABLE project*results (id SERIAL PRIMARY KEY, project*id INTEGER REFERENCES projects(id), heat*load FLOAT, -- kW annual*energy*demand FLOAT, -- kWh recommended*hp*capacity FLOAT, -- kW cop FLOAT, scop FLOAT, capex FLOAT, -- $ opex*annual FLOAT, -- $/year payback*years FLOAT, carbon*savings FLOAT, -- kg CO_2/year created_at TIMESTAMP DEFAULT NOW()); ```

4.2 Sample Data (Materials)

| ID | Name | Type | U-Value (W/m²·K) | Thermal Conductivity (W/m·K) | Density (kg/m³) | |----|------|------|------------------|------------------------------|-----------------| | 1 | Brick Wall (220mm) | Wall | 1.2 | 0.72 | 1700 | | 2 | Timber Frame (150mm) | Wall | 0.3 | 0.13 | 50 | | 3 | Double Glazing | Window | 2.8 | - | - | | 4 | Triple Glazing | Window | 1.6 | - | - | | 5 | Concrete Floor (200mm) | Floor | 1.5 | 1.7 | 2300 | | 6 | Insulated Roof (300mm) | Roof | 0.15 | 0.04 | 30 |

4.3 Sample Data (Heat Pump Models)

| ID | Name | Type | Min Capacity (kW) | Max Capacity (kW) | COP @ 7°C | COP @ 35°C | |----|------|------|-------------------|-------------------|-----------|-----------| | 1 | Generic ASHP (Small) | Air-Source | 5 | 10 | 3.5 | 2.0 | | 2 | Generic ASHP (Medium) | Air-Source | 10 | 20 | 3.8 | 2.2 | | 3 | Generic GSHP (Horizontal) | Ground-Source | 5 | 15 | 4.0 | 3.5 | | 4 | Generic GSHP (Vertical) | Ground-Source | 10 | 30 | 4.5 | 4.0 |

⚙ 5. Core Algorithms

5.1 Heat Loss Calculation (BS EN 12831 Simplified)

Formula: `Q = Σ (U × A × ΔT) + Q_ventilation + Q_infiltration` Where: - `Q` = Total heat loss (W) - `U` = U-value of surface (W/m²·K) - `A` = Area of surface (m²) - `ΔT` = Temperature difference (indoor - outdoor) (°C) - `Q_ventilation` = Heat loss from ventilation (W) - `Q_infiltration` = Heat loss from infiltration (W)

Steps: 1. Calculate **transmission heat loss** for each surface (walls, roof, floor, windows). 2. Calculate **ventilation heat loss**: `Q_vent = 0.33 × n × V × ΔT` - `n` = Air changes per hour (default: 0.5 for residential) - `V` = Volume of building (m³) 3. Sum all losses to get **total heat load (kW)**.

Example: - Building: 100 m², 2.5m height → Volume = 250 m³ - Walls: 200 m², U=0.3 W/m²·K - Roof: 100 m², U=0.2 W/m²·K - Windows: 20 m², U=2.8 W/m²·K - Outdoor temp: -5°C, Indoor temp: 20°C → ΔT = 25°C - Ventilation: 0.5 ACH `Q_walls = 0.3 × 200 × 25 = 1,500 W Q_roof = 0.2 × 100 × 25 = 500 W Q_windows = 2.8 × 20 × 25 = 1,400 W Q_vent = 0.33 × 0.5 × 250 × 25 = 1,031 W Total Q = 1,500 + 500 + 1,400 + 1,031 = 4,431 W ≈ 4.4 kW`

5.2 Heat Pump Sizing

Steps: 1. **Peak Heat Load** = `Q_total` (from heat loss calculation). 2. **Safety Factor** = 1.15 (15% oversizing for cold climates). 3. **Recommended Capacity** = `Q_total × Safety Factor`. 4. **Select Heat Pump** = Smallest model with `Capacity ≥ Recommended Capacity`.

Example: - Peak load = 4.4 kW - Recommended capacity = 4.4 × 1.15 = **5.06 kW** - Selected model: **Generic ASHP (Small, 5–10 kW)**

5.3 COP/SCOP Estimation

COP (Coefficient of Performance): - **Air-Source Heat Pump (ASHP):** `COP = 0.05 × T_outdoor + 2.5 (Simplified linear model)` - Example: At 7°C → COP = 0.05×7 + 2.5 = **2.85** (real-world: ~3.5) - At -5°C → COP = 0.05×(-5) + 2.5 = **2.25** - **Ground-Source Heat Pump (GSHP):** - COP = **4.0** (constant, as ground temp is stable).

SCOP (Seasonal COP): - Weighted average COP over the year (based on climate data). - **Formula:** `SCOP = Σ (COP_i × H_i) / Σ H_i` - `COP_i` = COP at temperature `i` - `H_i` = Heating degree hours at temperature `i`

Example (Simplified): | Month | Avg Temp (°C) | COP | Heating Degree Days (HDD) | Weighted COP |
|-------|--------------|-----|--------------------------|--------------| | Jan | 0 | 2.5 | 500 | 1,250 |
| Feb | 2 | 2.6 | 450 | 1,170 | | Mar | 5 | 2.85| 300 | 855 | | ... | ... | ... | ... | ... | | **Total** | - | - | **2,000** | **8,000** | | **SCOP** = 8,000 / 2,000 = **4.0**

5.4 Economic Analysis

CAPEX: - `Heat Pump Cost` = `$/kW × Capacity` (e.g., $1,500/kW) - `Installation Cost` = `$/kW × Capacity` (e.g., $1,000/kW) - **Total CAPEX** = Heat Pump Cost + Installation Cost

OPEX: - `Annual Energy Demand` = `Heat Load / COP × Runtime` (kWh) - `Annual Electricity Cost` = `Energy Demand × Electricity Price` ($/kWh)

Savings (vs. Gas Boiler): - `Gas Boiler Efficiency` = 0.9 (90%) - `Annual Gas Demand` = `Heat Load / 0.9 × Runtime` (kWh) - `Annual Gas Cost` = `Gas Demand × Gas Price` ($/kWh) - **Annual Savings** = Gas Cost - Electricity Cost

Simple Payback: `Payback (years) = CAPEX / Annual Savings`

Carbon Savings: `Carbon Savings (kg CO₂/year) = (Gas Demand × Carbon Intensity Gas) - (Electricity Demand × Carbon Intensity Electricity)`

🔌 6. API Specification

6.1 Endpoints

| Method | Endpoint | Description | Request Body | Response |
|--------|----------|-------------|--------------|----------| | `GET` | `/api/materials` | List all materials | - | `Material[]` | | `GET` | `/api/materials/{id}` | Get material by ID | - | `Material` | | `GET` | `/api/locations` | Search locations | `{query: string}` | `Location[]` | | `GET` | `/api/climate/{locationId}` | Get climate data | - | `ClimateData[]` | | `GET` | `/api/heat-pumps` | List heat pump models | - | `HeatPumpModel[]` | | `GET` | `/api/economic/{country}` | Get economic data

| - | EconomicData | | POST | /api/projects | Create project | ProjectInput | Project | | POST | /api/projects/{id}/calculate | Run simulation | SimulationInput | SimulationResult |

6.2 Request/Response Examples

POST /api/projects/{id}/calculate

Request: json { "building": { "length": 10.0, "width": 8.0, "height": 2.5, "floors": 1, "orientation": "S" }, "surfaces": [{ "type": "wall", "material_id": 1, "area": 50.0, "thickness": 0.22 }, { "type": "roof", "material_id": 5, "area": 80.0, "thickness": 0.2 }], "windows": [{ "glazing_type": "double", "area": 15.0, "u_value": 2.8 }], "location": { "city": "London", "country": "UK", "latitude": 51.5074, "longitude": -0.1278 }, "indoor_temp": 20.0, "heat_pump_type": "air-source", "existing_system": "gas-boiler" }

Response: json { "heat_load": 4.43, // kW "annual_energy_demand": 12000.0, // kWh "recommended_hp": { "model_id": 1, "name": "Generic ASHP (Small)", "capacity": 5.0, // kW "cop": 3.5, "scop": 4.0 }, "economics": { "capex": 7500.0, // $ "opex_annual": 800.0, // $/year "payback_years": 5.2, "carbon_savings": 2500.0 // kg CO_2/year }, "heat_loss_breakdown": { "walls": 1.5, // kW "roof": 0.5, // kW "windows": 1.4, // kW "ventilation": 1.03 // kW } }

🎨 7. UI/UX Design

7.1 Wireframes

1. Homepage / Landing

+-------------------------------------+ | [Logo] HeatPumpSim | | "Feasibility & Simulation in Minutes"| | | | [Get Started] [Learn More] | | | | +----------------------------+ | | | 1. Input Building Details | | | | 2. Select Materials | | | | 3. Get Instant Results | | | +----------------------------+ | | | | [Example Report] [Testimonials] | +-------------------------------------+

2. Input Workflow (Step 1: Building)

+-------------------------------------+ | [Step 1 of 4] Building Dimensions | | | | Length (m): [____] | | Width (m): [____] | | Height (m): [____] | | Floors: [____] | | Orientation: [N ▼] | | | | | [Back] [Next] | +-------------------------------------+

3. Input Workflow (Step 2: Materials)

```
+-------------------------------------+ | [Step 2 of 4] Envelope Materials | | | | +---------------------------+ | | | Surface Type: [Wall ▼] | | | | Material: [Brick ▼] | | | | Area (m²): [____] | | | | Thickness (m): [0.22] | | | | [Add Surface] | | | | +---------------------------+ | | | | Existing Surfaces: | | - Wall: Brick (220mm), 50m² | | - Roof: Concrete (200mm), 80m² | | | | [Back] [Next] | +-------------------------------------+
```

4. Input Workflow (Step 3: Location & Settings)

```
+-------------------------------------+ | [Step 3 of 4] Location & Settings | | | | City: [London ▼] | | Country: [UK ▼] | | Indoor Temp: [20 °C] | | Heat Pump Type: [Air-Source ▼] | | Existing System: [Gas Boiler ▼] | | | | [Back] [Next] | +-------------------------------------+
```

5. Results Dashboard

```
+-------------------------------------+ | [Results] [Save] [Export PDF] | | | | +---------------------------+ | | | Recommended Heat Pump: | | | | **Generic ASHP (Small)** | | | | Capacity: 5 kW | | | | COP: 3.5 | SCOP: 4.0 | | | +---------------------------+ | | | | +---------------------------+ | | | Heat Load: 4.43 kW | | | | Annual Energy: 12,000 kWh | | | +---------------------------+ | | | | +---------------------------+ | | | Economics: | | | | CAPEX: $7,500 | | | | Annual OPEX: $800 | | | | Payback: 5.2 years | | | | Carbon Savings: 2,500 kg/year | | | +---------------------------+ | | | | [Heat Loss Breakdown Chart] | | (Pie chart: Walls 34%, Roof 11%, ...)| | | | [COP vs. Temperature Chart] | | (Line chart: COP drops as temp ↓) | +-------------------------------------+
```

6. Heat Loss Breakdown Chart

```
mermaid pie title Heat Loss Breakdown "Walls" : 34 "Roof" : 11 "Windows" : 32 "Ventilation" : 23
```

7. COP vs. Temperature Chart

```
mermaid lineChart title COP vs. Outdoor Temperature xAxis Outdoor Temp (°C) yAxis COP line ASHP, 2.5, 2.85, 3.2, 3.5, 3.8 line GSHP, 4.0, 4.0, 4.0, 4.0, 4.0
```

🛠 8. Technical Stack

| Category | Technology | Justification | |----------|------------|---------------| | **Frontend** | Next.js (React) | SEO-friendly, SSR, easy deployment | | **Styling** | Tailwind CSS | Rapid UI development, responsive | | **State Management** | Zustand | Lightweight, simple API | | **Forms** | React Hook Form | Validated, performant forms | | **Charts** | Plotly.js | Interactive, scientific-grade | | **Backend** | FastAPI (Python) | Fast, async, OpenAPI docs | | **Database** | PostgreSQL | Relational, scalable | | **ORM** | SQLAlchemy | Pythonic, flexible | | **Auth** | JWT (Future) | Stateless, secure | | **Deployment** | Vercel (Frontend) + Railway (Backend) | Easy, scalable | | **CI/CD** | GitHub Actions | Automated testing/deployment | | **Testing** | pytest (Backend) + Jest (Frontend) | Comprehensive coverage | | **Monitoring** | Sentry | Error tracking |

📁 9. Project Structure

```
heat-pump-sim/ ├── frontend/ # Next.js App │ ├── public/ │ ├── src/ │ │ ├── components/ # Reusable UI components │ │ │ ├── inputs/ │ │ │ ├── results/ │ │ │ └── layout/ │ │ ├── pages/ # Next.js pages │ │ │ ├── index.tsx # Landing │ │ │ ├── project/ │ │ │ │ ├── [id].tsx │ │ │ │ └── new.tsx │ │ │ └── api/ # API routes (if needed) │ │ ├── styles/ # Tailwind + custom CSS │ │ ├── hooks/ # Custom React hooks │ │ ├── utils/ # Helper functions │ │ ├── types/ # TypeScript types │ │ └── store/ # Zustand state │ ├── package.json │ └── next.config.js │ ├── backend/ # FastAPI │ ├── app/ │ │ ├── main.py # FastAPI app │ │ ├── api/ # API routes │ │ │ ├── projects.py │ │ │ ├── materials.py │ │ │ └── ... │ │ ├── core/ # Core logic │ │ │ ├── heat_loss.py │ │ │ ├── heat_pump.py │ │ │ └── economics.py │ │ ├── db/ # Database │ │ │ ├── models.py # SQLAlchemy models │ │ │ └── session.py # DB session │ │ └── schemas.py # Pydantic models │ ├── tests/ # pytest tests │ ├── requirements.txt │ └── Dockerfile │ ├── data/ # Static data (seed DB) │ ├── materials.json │ ├── climate.json │ └── heat_pumps.json │ ├── docs/ # Documentation │ ├── API.md │ └── USER_GUIDE.md │ └── README.md
```

🧪 10. Testing Strategy

10.1 Unit Tests (Backend)

| Module | Test Cases | Coverage | |--------|------------|----------| | `heat_loss.py` | U-value calculations, surface area, ΔT | 100% | | `heat_pump.py` | Sizing, COP interpolation, model selection | 100% | | `economics.py` | CAPEX, OPEX, payback, carbon savings | 100% | | `api/projects.py` | Input validation, error handling | 90% |

Example Test (pytest): ```python

tests/test*heat*loss.py

from app.core.heat*loss import calculate*heat_loss

def test*heat*loss*walls(): surfaces = [{*"*u*value": 0.3, "area": 200, "delta*t*": 25}] *ventilation = {*"*air*changes": 0.5, "volume": 250, "delta*t*": 25} result = *calculate*heat_loss(surfaces, ventilation) assert result["total"] == pytest.approx(1500 + 1031.25, rel=0.01) # 1500 (walls) + 1031.25 (vent)
```

### 10.2 Integration Tests

| Scenario | Test | Expected | |----------|------|----------| | Full workflow | Input → Calculate → Results | Valid JSON response | | Invalid input | Missing fields | 422 Unprocessable Entity | | Database error | SQL failure | 500 Internal Server Error |

### 10.3 Frontend Tests

| Component | Test | Tool | |-----------|------|------| | Input forms | Validation, submission | Jest + React Testing Library | | Charts | Data rendering | Jest | | Responsive design | Mobile/desktop layouts | Cypress |

### 10.4 Validation Strategy

1. **Benchmark Against EnergyPlus/HPDM**
   - Compare results for **standard test cases** (e.g., reference buildings).
   - Target: **±5% accuracy**.
2. **User Testing**
   - Recruit **5–10 HVAC contractors** for feedback.
   - Track **time-to-completion** and **error rates**.
3. **Edge Cases**
   - Extremely cold climates (-20°C).
   - Highly insulated buildings (Passivhaus).
   - Large commercial buildings (10,000 m²).

---

## 11. Deployment Plan

### 11.1 Environment Setup

| Environment | Frontend | Backend | Database | |-------------|----------|---------|----------| | **Development** | `localhost:3000` | `localhost:8000` | Local PostgreSQL | | **Staging** | Vercel Preview | Railway | Railway PostgreSQL | | **Production** | Vercel | Railway | Railway PostgreSQL |
```

11.2 CI/CD Pipeline (GitHub Actions)

```yaml

# .github/workflows/test-and-deploy.yml

name: Test and Deploy

on: push: branches: [main, develop] pull_request: branches: [main]

jobs: test-backend: runs-on: ubuntu-latest steps: - uses: actions/checkout@v4 - uses: actions/setup-python@v4 with: python-version: "3.10" - run: pip install -r backend/requirements.txt - run: pytest backend/tests/

test-frontend: runs-on: ubuntu-latest steps: - uses: actions/checkout@v4 - uses: actions/setup-node@v3 with: node-version: 18 - run: npm install - run: npm test working-directory: frontend

deploy-staging: needs: [test-backend, test-frontend] if: github.ref == 'refs/heads/develop' runs-on: ubuntu-latest steps: - uses: actions/checkout@v4 - run: vercel --token ${{ secrets.VERCEL*TOKEN }} --env staging working-directory: frontend - run: railway up --detach working-directory: backend env: RAILWAY*TOKEN: ${{ secrets.RAILWAY_TOKEN }}

deploy-production: needs: [test-backend, test-frontend] if: github.ref == 'refs/heads/main' runs-on: ubuntu-latest steps: - uses: actions/checkout@v4 - run: vercel --token ${{ secrets.VERCEL*TOKEN }} --prod working-directory: frontend - run: railway up --detach --environment production working-directory: backend env: RAILWAY*TOKEN: ${{ secrets.RAILWAY_TOKEN }}
```

11.3 Monitoring

| Tool | Purpose | |------|---------| | **Sentry** | Error tracking (frontend + backend) | | **Vercel Analytics** | Frontend performance | | **Railway Metrics** | Backend performance | | **PostgreSQL Logs** | Database queries |

□ 12. MVP Development Timeline

| Week | Task | Owner | Status | |------|------|-------|--------| | 1–2 | **Project Setup** | Dev Team | □ | | | - Define tech stack | | | | | - Set up repos (frontend/backend) | | | | | - Configure CI/CD | | | | 3–4 | **Database Design** | Backend | □ | | | - Schema design | | | | | - Seed data (materials, climate, heat pumps) | | | | | - ORM setup (SQLAlchemy) | | | | 5–6 | **Backend Core** | Backend | □ | | | - Heat loss calculation | | | | | - Heat pump sizing | | | | | - Economic analysis | | | | 7–8 | **API Development** | Backend | □ | | | - RESTful endpoints | | | | | - Input validation | | | | | - Error handling | | | | 9–10 | **Frontend UI** | Frontend | □ | | |

- Landing page | | | | | - Input workflow (4 steps) | | | | | - Results dashboard | | | | 11–12 | **Frontend Logic** | Frontend | □ | | | - Form handling | | | | | - API integration | | | | | - State management | | | | 13–14 | **Testing** | QA | □ | | | - Unit tests (backend) | | | | | - Integration tests | | | | | - User testing (beta) | | | | 15–16 | **Bug Fixes & Polish** | Dev Team | □ | | | - Fix critical issues | | | | | - Optimize performance | | | | | - Final UX tweaks | | | | 17–18 | **Deployment** | DevOps | □ | | | - Staging deployment | | | | | - Production deployment | | | | | - Monitoring setup | | | | 19–20 | **Documentation** | Tech Writer | □ | | | - User guide | | | | | - API docs (Swagger) | | | | | - README | | | | 21–24 | **Buffer** | - | □ | | | - Contingency time | | |

Total MVP Duration: 24 Weeks (6 Months)

□ Appendix: Sources & References

- **Market Data**: Grand View Research, Market Research Future
- **Existing Tools**: GeoT*SOL, Polysun, EnergyPlus, HPDM, TESPy
- **Standards**: BS EN 12831, ASHRAE 90.1
- **Greenhouse/CEA**: Appropedia, PMC/NCBI, ScienceDirect